ECE 220 Computer Systems & Programming

Lecture 7 — Introduction to Functions in C

September 15, 2026

Prof. Sayan Mitra mitras@illinois.edu Office hours: TBD
Course website: courses.grainger.illinois.edu/ece220/fa2026

ILLINOIS
Electrical & Computer Engineering
GRAINGER COLLEGE OF ENGINEERING

The Reuse Problem, Revisited

Subroutines are back — with names, types, and no register juggling.

```
; args in R0, R1 (by convention)
; save R7!  callee-save R3!
      JSR  MULT
; result in R2 (by convention)
```

LC-3 subroutine call (Lecture 2)


```
answer = Fact(n);
```

C function call (today)

Today's Two Questions

A function is a contract:
typed inputs, one typed output.

1. How does data get *into* and *out of* a C function?
2. What happens to *your* variables while someone else's function runs?

Question 2 has a one-word spoiler — copies — and a famous victim: `swap`.

Prototype, Call, Definition

```c
/* answer = Fact(number): prototype, call, definition */
#include <stdio.h>

int Fact(int n);              /* PROTOTYPE: ends with ; */

int main() {
    int number, answer;
    printf("Enter a number: ");
    scanf("%d", &number);
    answer = Fact(number);    /* the CALL */
    printf("factorial of %d is %d\n", number, answer);
    return 0;
}

int Fact(int n) {             /* the DEFINITION */
    int i, result = 1;        /* locals: born at the call */
    for (i = 1; i <= n; i++)
        result = result * i;
    return result;            /* typed return value */
}
```

- **Prototype**: tells the compiler the _____ — of the arguments and the return value — before the first call; ends with _____
- **Call**: `Fact(number)` is an *expression* with a value
- **Definition**: the code; may appear before or after `main` (before: no prototype needed)

Inside the Definition

```
int Fact(int n) {                /* the DEFINITION */
    int i, result = 1;           /* locals: born at
     the call */
    for (i = 1; i <= n; i++)
        result = result * i;
    return result;               /* typed return value
      */
}
```

In-class: what does `Fact(4)` return? _____

- `n`, `i`, `result` are **local**: born at the call, gone after `return`
- `return` does two jobs: hands back a value *and* ends the function
- What is `i` after the return?

_____

Functions You Already Use

- `printf`, `scanf`, `getchar` — library functions, declared in `<stdio.h>`
- They *return values* you have been ignoring: `scanf` returns the _____ — checking it catches bad input
- `void` return type = nothing comes back; `f(void)` or `f()` = nothing goes in
- `man 3 printf` shows any library function's full contract

Every function call in every C program follows today's rules — including `main` itself, called by the operating system.

Arguments Are Copies (Pass by Value)

```c
/* Arguments are COPIES (pass by value) */
#include <stdio.h>

void tryToChange(int a) {
    a = 99;                    /* changes the copy
     */
    printf("inside:  a = %d\n", a);
}

int main() {
    int x = 5;
    tryToChange(x);
    printf("outside: x = %d\n", x);
    return 0;
}
```

In-class — predict the output:

inside: `a` = _____

outside: `x` = _____

- The call copies the *value* of `x` into a brand new variable `a`
- Nothing the callee does to its copy can touch the caller's original

Why Bother? Decomposition

The same menu program, two ways (from I. Abraham's `mat_nofun.c` / `mat_fun.c`):

Without functions:

- one 50-line `main`
- the same nested printing loop pasted into two `switch` cases
- a change to the printing means editing both copies

With functions:

- `case 'u': print_upper(N); break;`
- `main` reads as what it does: validate, ask, dispatch
- each piece testable on its own

Same reasons we wrote LC-3 subroutines — abstraction, reuse, separable development — now with the compiler enforcing the contract.

In-Class Problem: Write swap

Fill in `swap` so it exchanges `x` and `y` — then predict the output.

- Does it print `x = 3, y = 2`?

  _____

- Why?

To fix swap we must find out where x and y actually *live*. Thursday: the run-time stack.

```c
/* Does this swap main's x and y? */
#include <stdio.h>

void swap(int x, int y) {
    /* declare a temporary */

    /* three assignments to exchange x and y */

}

int main() {
    int x = 2, y = 3;
    swap(x, y);
    printf("x = %d, y = %d\n", x, y);
    return 0;
}
```

Summary & What's Next

Today

- **Anatomy**: prototype (types, before first use), call (an expression), definition (the code)
- **The contract**: zero or more typed arguments in, at most one typed value out; locals are born at the call and die at return
- **Pass by value**: arguments are *copies* — which is why `swap` silently fails

Next lecture

- Where the copies live: activation records, R5/R6, and the full C calling convention in LC-3

Code from today: `fact.c`, `copy.c`, `swap.c`.